Introduction to Deep Learning

&&

Харитон Пицик

2026-03-25

Клуб машинного обучения

2000е годы. Революция ML

&&

В 2005-2006 годах группы исследователей под руководством Джеффри Хинтона в ун-те Торонто и Йошуа Бенджи в ун-те Монреаля научились обучать *глубокие нейронные сети*, что стало революцией в мире машинного обучения.

2000е годы. Революция ML

&&

В 2005-2006 годах группы исследователей под руководством Джеффри Хинтона в ун-те Торонто и Йошуа Бенджи в ун-те Монреаля научились обучать *глубокие нейронные сети*, что стало революцией в мире машинного обучения.

С тех пор в самых разнообразных доменах наилучшие результаты получают именно благодаря им. Одним из первых громких промышленных успехов стал алгоритм распознавания речи, разработанный всё тем же Хинтоном. С тех пор любой помощник вроде Apple Siri работает исключительно на глубоких НН.

Что стало катализатором революции?

&&

Что стало катализатором революции?

1. Предобучение с учителем

Что стало катализатором революции?

1. Предобучение с учителем
2. Данных стало больше

Что стало катализатором революции?

&&

1. Предобучение с учителем
2. Данных стало больше
3. Прогресс в вычислительных мощностях

Считается, что ИИ как наука началась с *теста Тьюринга*, в 1950 году.

Считается, что ИИ как наука началась с *теста Тьюринга*, в 1950 году.

Идея: алгоритм и ещё 2 человека играли в “судью”, где “ИИ” предлагалось имитировать кого-то. Задача судьи – распознать ИИ.

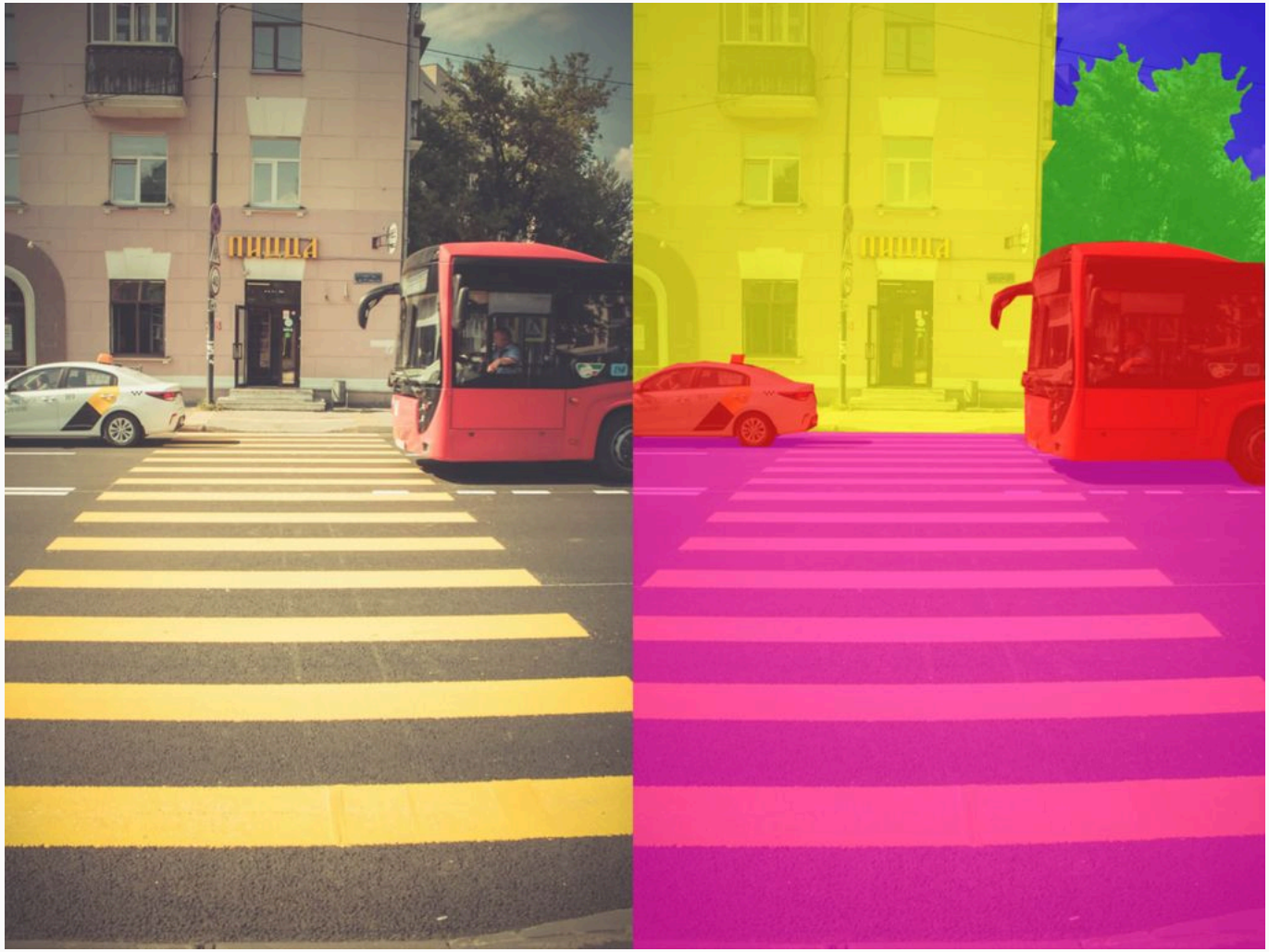
Что умеют нейронные сети?

Что умеют нейронные сети? &&

1. Сегментация изображений

Что умеют нейронные сети?

&&

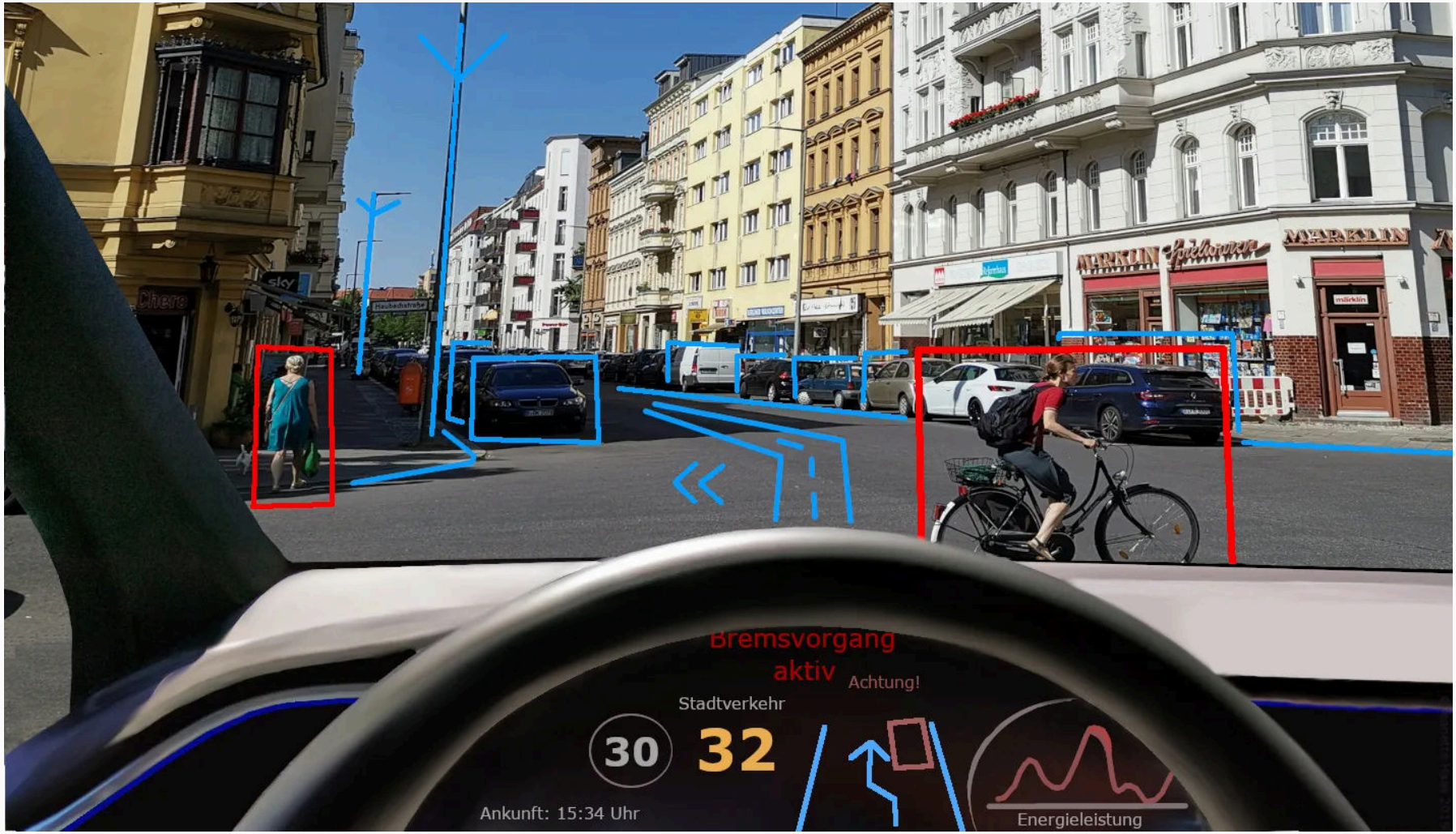


Что умеют нейронные сети? &&

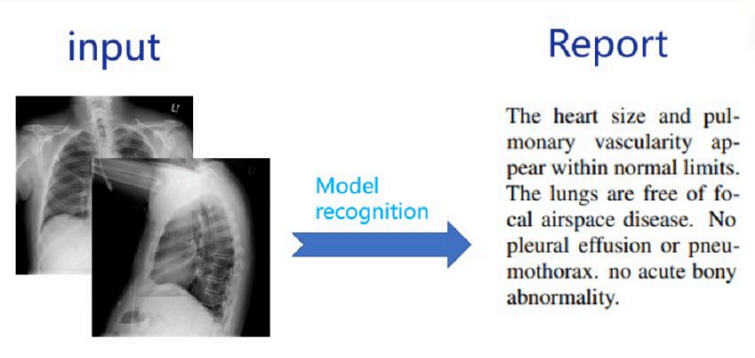
2. Детекция на изображениях

Что умеют нейронные сети?

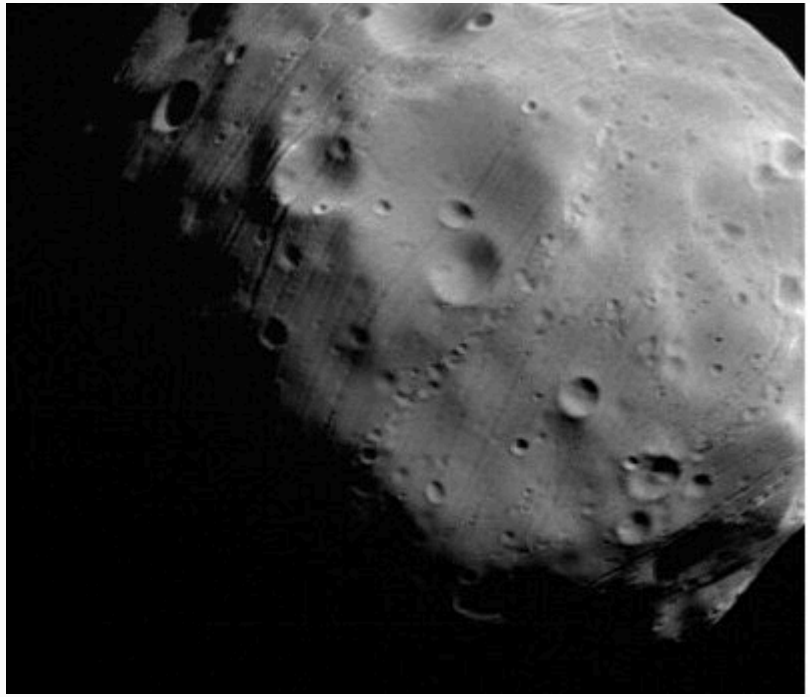
&&

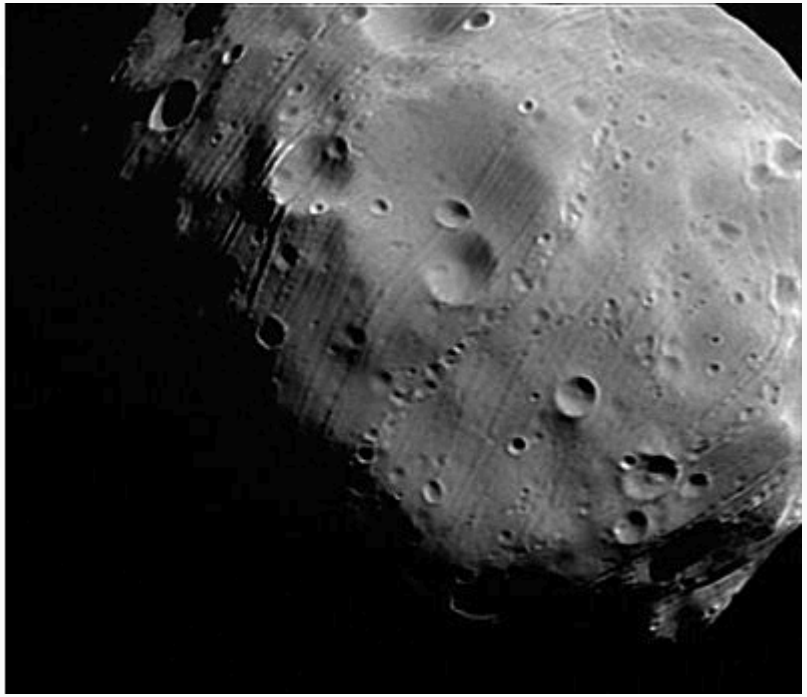


3. Описание по фото



4. Генерация изображений





Что умеют нейронные сети? &&

5. Суммаризация текста
6. Построение системы чат-бота
7. Машинный перевод
8. Генерация текста
9. Сопоставить текст с автором

Граф вычислений, дифференцирование на нём.

Основное предположение: если от каждой простой части сложной функции удаётся взять производную, то производную можно посчитать и от исходной функции.

Граф вычислений, дифференциирование на нём. &&

Основное предположение: если от каждой простой части сложной функции удаётся взять производную, то производную можно посчитать и от исходной функции.

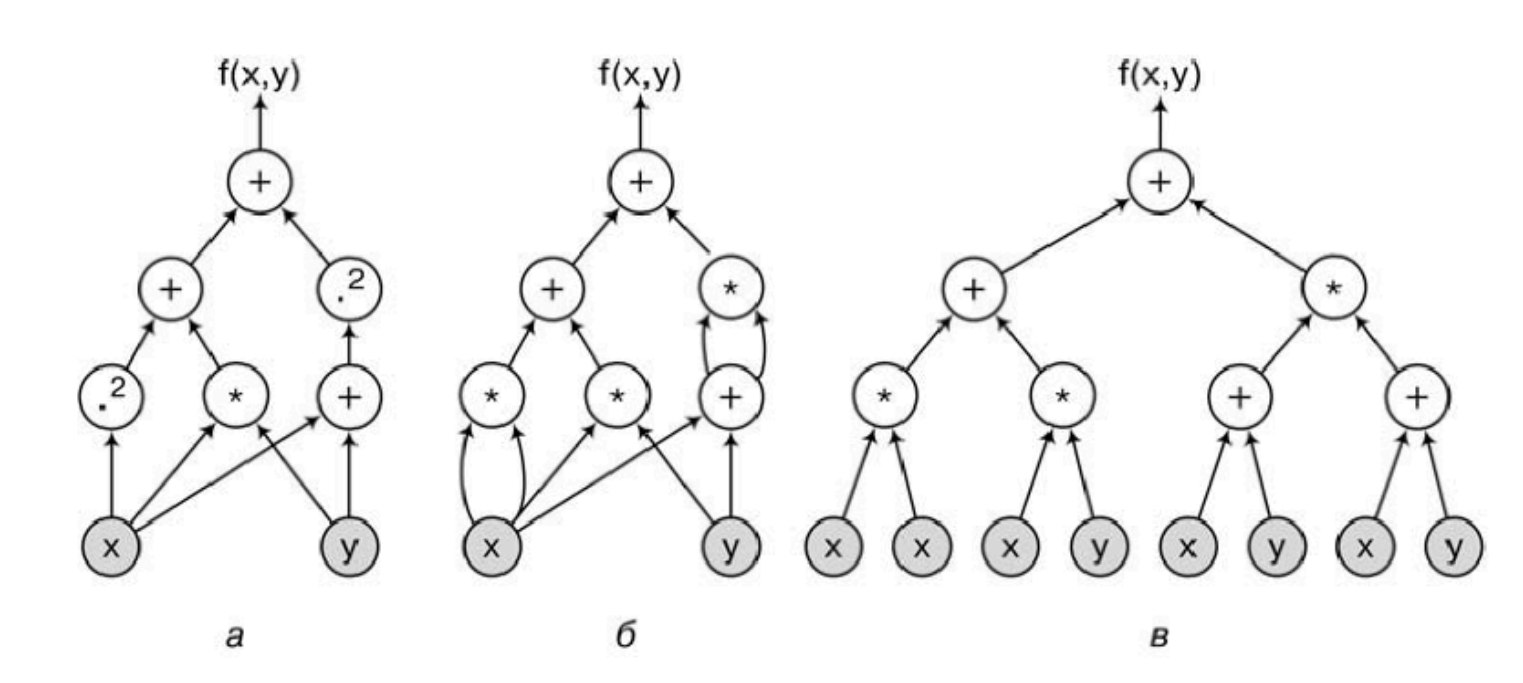
Самый простой способ представить композицию – это представление в виде графа. Граф вычислений – это граф, узлами которого являются функции, а ребра связывают функции со своими аргументами.

Рассмотрим пример:

$$f(x, y) = x^2 + xy + (x + y)^2$$

Рассмотрим пример:

$$f(x, y) = x^2 + xy + (x + y)^2$$



Теперь мы знаем, что сколь угодно сложную функцию можно разложить в граф, и если от каждого узла можно найти производную, то производную можно посчитать и от исходной функции. Давайте вспомним ещё одно занимательное правило из математики:

Теперь мы знаем, что сколь угодно сложную функцию можно разложить в граф, и если от каждого узла можно найти производную, то производную можно посчитать и от исходной функции. Давайте вспомним ещё одно занимательное правило из математики:

$$(f \circ g)'(x) = (f(g(x)))' = f'(g(x))g'(x)$$

Теперь мы знаем, что сколь угодно сложную функцию можно разложить в граф, и если от каждого узла можно найти производную, то производную можно посчитать и от исходной функции. Давайте вспомним ещё одно занимательное правило из математики:

$$(f \circ g)'(x) = (f(g(x)))' = f'(g(x))g'(x)$$

Из этого следует, что если f зависит от g, а g зависит от x, то маленькое изменение параметра x на δx приведет к маленькому изменению $g(x)$ примерно на $\delta g = g'(x)\delta x$. Оно же в свою очередь изменит $f(g(x))$ примерно на $\delta f = f'(g(x))\delta g = f'(g(x))g'(x)\delta x$

Единственный разумный шаг, который нам нужно добавить – функции могут быть векторными, т.е. принимать вектор в качестве аргумента. В таком случае речь идёт не о производной, а о *градиенте*, векторе частных производных:

$$\nabla_x (f \circ g) = \frac{\partial f}{\partial g} \nabla_x g$$

Единственный разумный шаг, который нам нужно добавить – функции могут быть векторными, т.е. принимать вектор в качестве аргумента. В таком случае речь идёт не о производной, а о *градиенте*, векторе частных производных:

$$\nabla_x(f \circ g) = \frac{\partial f}{\partial g} \nabla_x g$$

Если g также будет являться вектором, то выходит, что f зависит от x не в одном месте, а сразу в нескольких: $f = (g_1(x), g_2(x), ..., g_k(x))$. Значит, приращение δx превратится в несколько разных приращений вида $\delta g_1, ..., \delta g_k$, каждое из которых внесёт разный вклад в приращение функции f, т.е.:

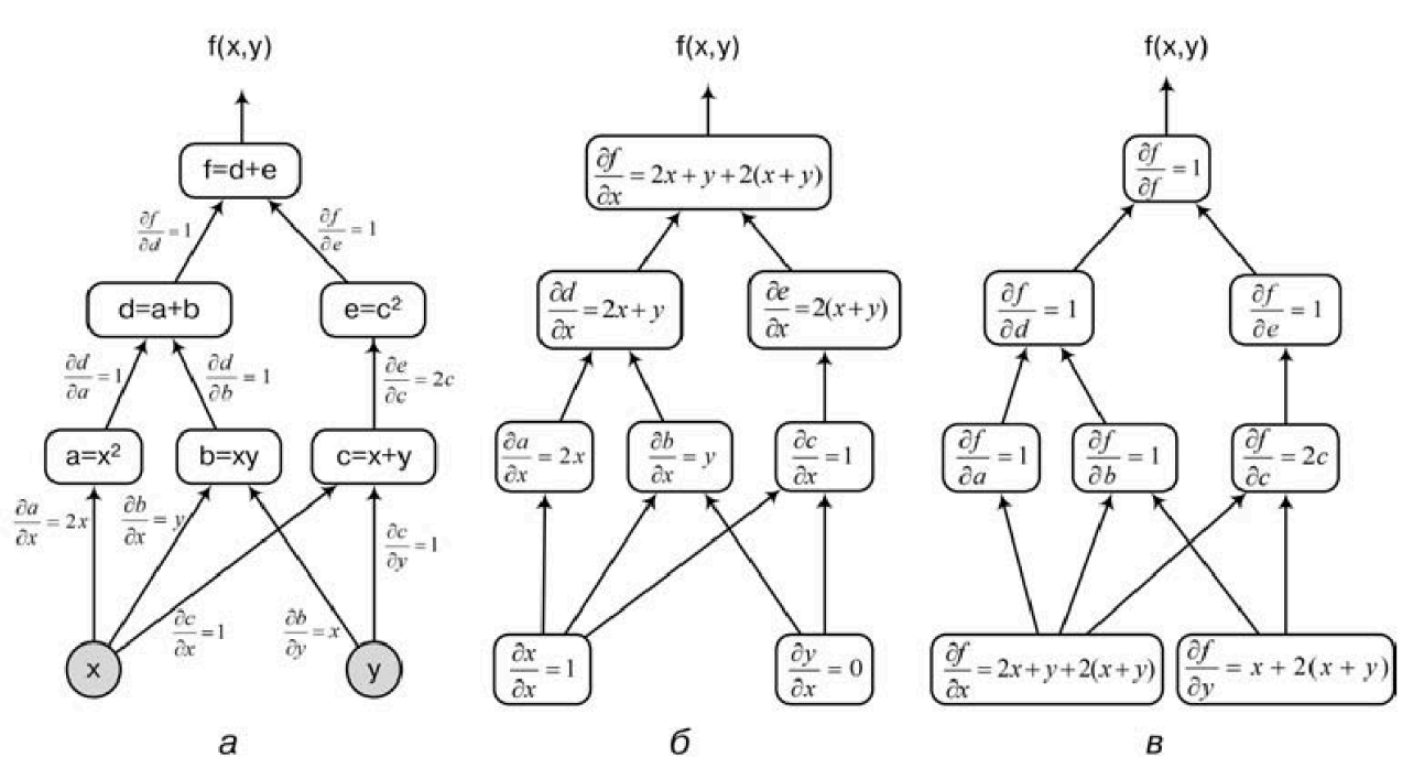
Единственный разумный шаг, который нам нужно добавить – функции могут быть векторными, т.е. принимать вектор в качестве аргумента. В таком случае речь идёт не о производной, а о *градиенте*, векторе частных производных:

$$\nabla_x(f \circ g) = \frac{\partial f}{\partial g} \nabla_x g$$

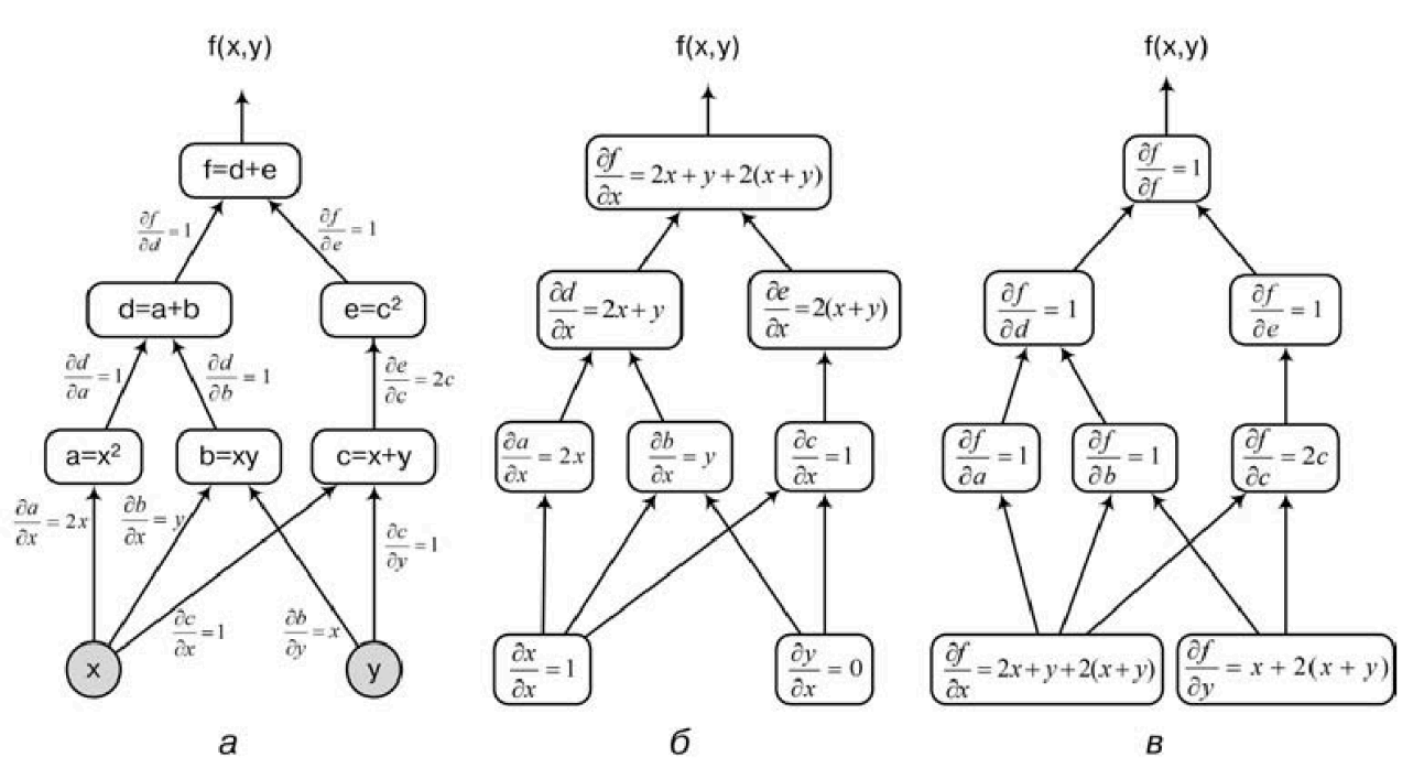
Если g также будет являться вектором, то выходит, что f зависит от x не в одном месте, а сразу в нескольких: $f = (g_1(x), g_2(x), ..., g_k(x))$. Значит, приращение δx превратится в несколько разных приращений вида $\delta g_1, ..., \delta g_k$, каждое из которых внесёт разный вклад в приращение функции f, т.е.:

$$\delta f = \frac{\partial f}{\partial g_1} \delta g_1 + ... + \frac{\partial f}{\partial g_k} \delta g_k$$

Граф вычислений, дифференциирование на нём. &&



Граф вычислений, дифференциирование на нём. &&



$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial x} + \frac{\partial f}{\partial c}\frac{\partial c}{\partial x} = 2x + y + 2(x + y)$$

Полученный алгоритм называется *обратным распространением ошибки* (backpropagation, backprop). Данный алгоритм является ключевым для обучения любых глубоких сетей, и любая архитектура упирается именно в него.

Полученный алгоритм называется *обратным распространением ошибки* (backpropagation, backprop). Данный алгоритм является ключевым для обучения любых глубоких сетей, и любая архитектура упирается именно в него.

Таким образом:

1. Любая функция дифференциируема, если узлы её графа дифференциируемы;

Полученный алгоритм называется *обратным распространением ошибки* (backpropagation, backprop). Данный алгоритм является ключевым для обучения любых глубоких сетей, и любая архитектура упирается именно в него.

Таким образом:

1. Любая функция дифференциируема, если узлы её графа дифференциируемы;
2. На графе возможно применить распространение градиента обратно, чтобы откорректировать веса.

Перцептрон Розенблатта

В своей сути, перцептрон Розенблатта является линейной моделью классификации, а именно – *бинарной*.

В своей сути, перцептрон Розенблатта является линейной моделью классификации, а именно – *бинарной*.

$$sign(w^T x) = sign(w_0 + w_1 x_1 + w_2 x_2 + ... + w_d x_d)$$

В качестве функции потерь закрепим т.н. *критерий перцептрона*:

В качестве функции потерь закрепим т.н. *критерий перцептрона*:

$$E_{P(w)} = -\sum_{x \in M} y(x)(w^T x)$$

при $y = \{-1, 1\}$

В качестве функции потерь закрепим т.н. *критерий перцептрона*:

$$E_{P(w)} = -\sum_{x \in M} y(x)(w^T x)$$

при $y = \{-1, 1\}$

где M – множество примеров, которые перцептрон классифицирует неверно. Иначе говоря, мы минимизируем количество отклонений от правильных ответов.

Почему нам недостаточно критерия перцептрона?

Почему нам недостаточно критерия перцептрона?

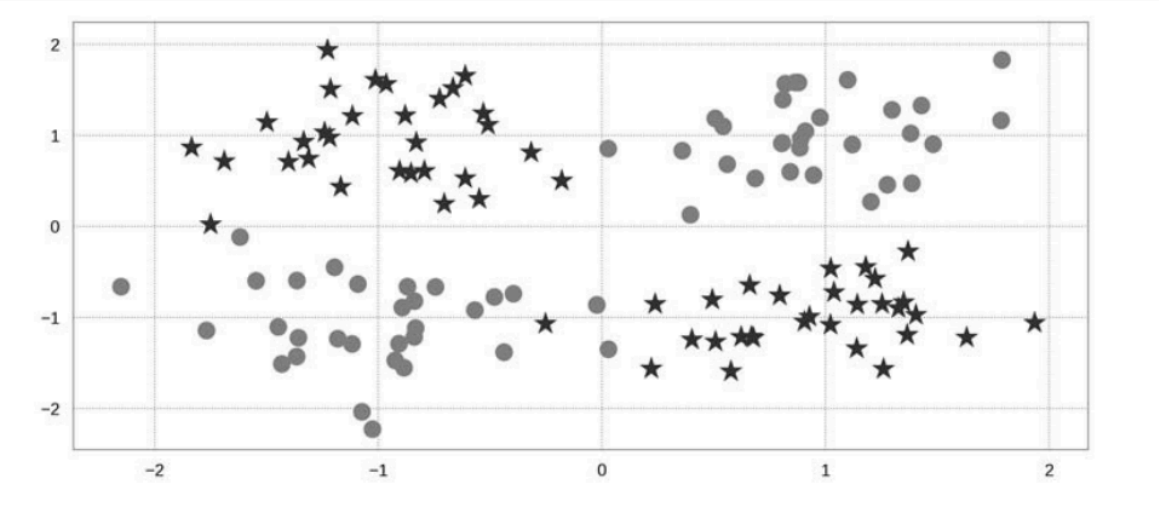
- Функция $sign(x)$ не дифференцируема в нуле и имеет нулевой градиент в остальных точках;

Почему нам недостаточно критерия перцептрона?

- Функция $sign(x)$ не дифференцируема в нуле и имеет нулевой градиент в остальных точках;
- Мы не можем использовать эффективные методы оптимизации (тот же градиентный спуск).

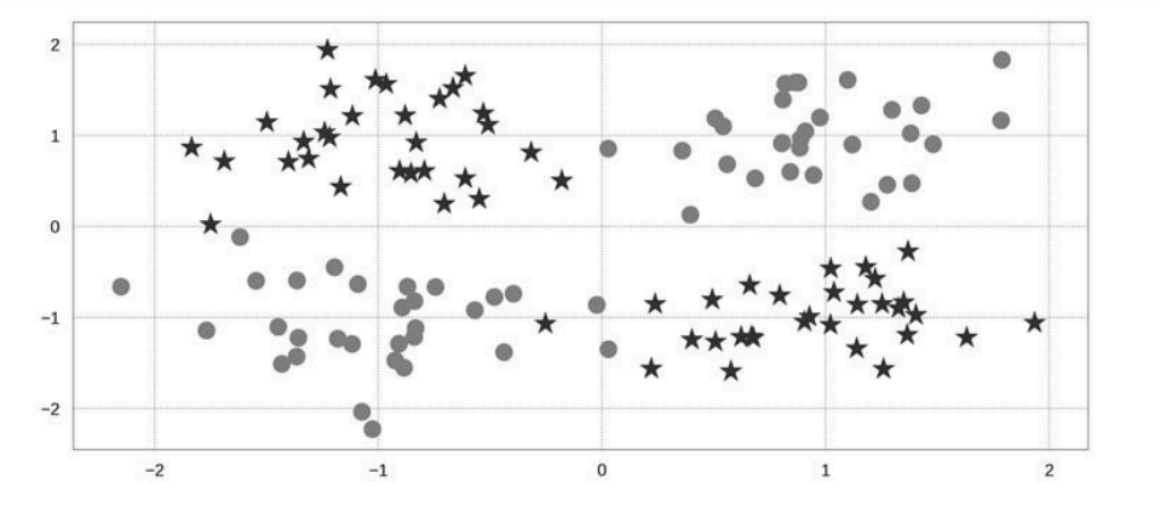
Почему нам недостаточно критерия перцептрона?

- Функция $sign(x)$ не дифференцируема в нуле и имеет нулевой градиент в остальных точках;
- Мы не можем использовать эффективные методы оптимизации (тот же градиентный спуск).



Почему нам недостаточно критерия перцептрона?

- Функция $sign(x)$ не дифференцируема в нуле и имеет нулевой градиент в остальных точках;
- Мы не можем использовать эффективные методы оптимизации (тот же градиентный спуск).



Решение – предсказывать более гладкую функцию, перейти к нелинейности.

Но это не всё – мы хотим знать уверенность модели в этом классе (т.е. вероятность, которую она выдаёт). Для этого нам нужно добавить сигмоиду:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Но это не всё – мы хотим знать уверенность модели в этом классе (т.е. вероятность, которую она выдаёт). Для этого нам нужно добавить сигмоиду:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

в контексте нейронных сетей подобные функции будем называть *функциями активации.*

Но это не всё – мы хотим знать уверенность модели в этом классе (т.е. вероятность, которую она выдаёт). Для этого нам нужно добавить сигмоиду:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

в контексте нейронных сетей подобные функции будем называть *функциями активации.*

В дальнейшем такую мини-модель мы будем использовать как базовую единицу для составления более сложных архитектур:

вход $\rightarrow$ линейная комбинация $\rightarrow$ функция активации

Но это не всё – мы хотим знать уверенность модели в этом классе (т.е. вероятность, которую она выдаёт). Для этого нам нужно добавить сигмоиду:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

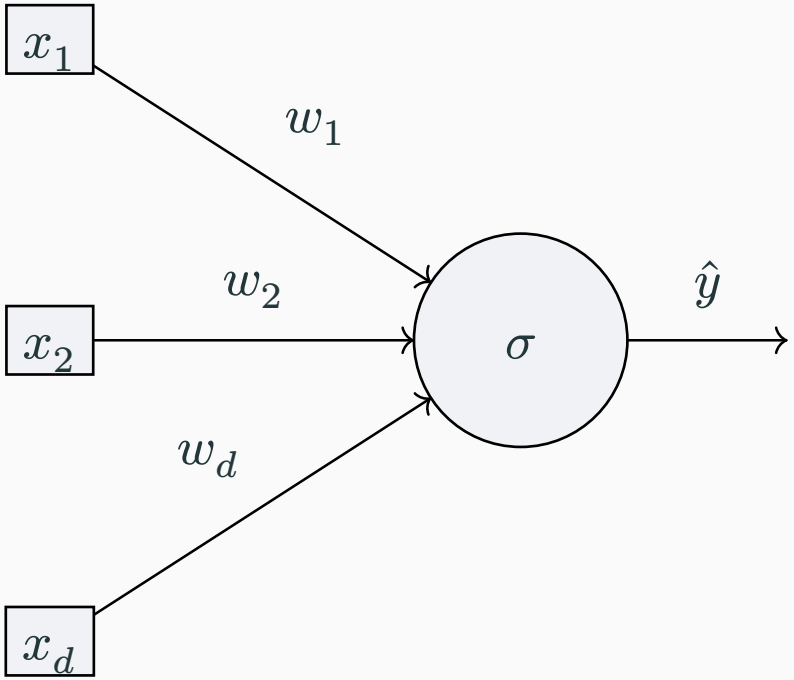
в контексте нейронных сетей подобные функции будем называть *функциями активации.*

В дальнейшем такую мини-модель мы будем использовать как базовую единицу для составления более сложных архитектур:

вход $\rightarrow$ линейная комбинация $\rightarrow$ функция активации

Формально, мы только что переизобрели **логистическую регрессию**, но посмотрели на проблему под немного другим углом.

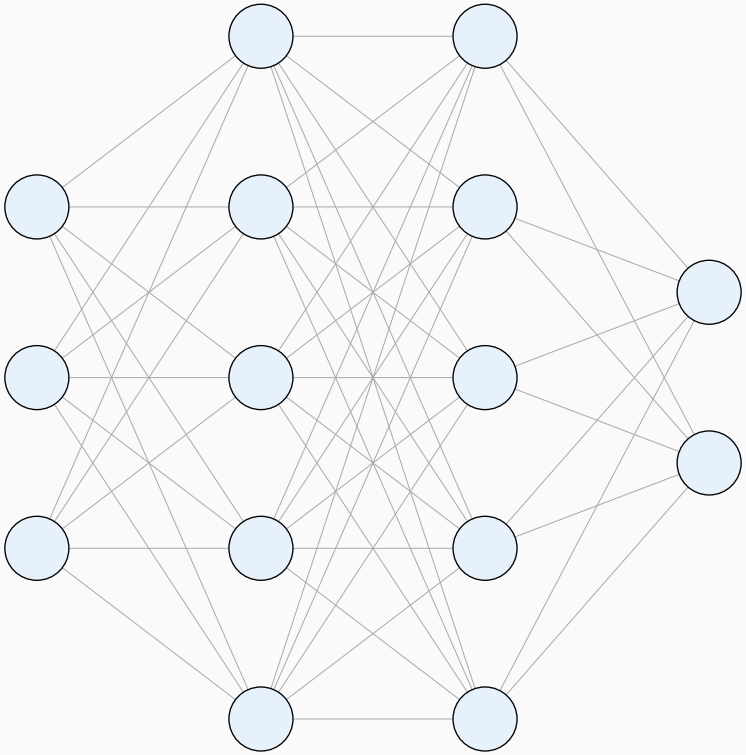
Архитектура MLP



Это – **логистическая регрессия**

Архитектура MLP

&&



А это – **многослойный перцептрон** (MultiLayer Perceptron)

Первый слой называется входным (input layer), последний слой называется выходным (output layer). Всё что между ними – скрытые слои (hidden state).

Векторизация: упрощаем сложное

&&

Если в слое k нейронов, и веса у них $w_1, w_2, ..., w_k$, где $w_i = (w_{i1}, w_{i2}, ..., w_{in})^T$, а на вход подаётся вектор $x = (x_1, x_2, ..., x_n)^T$, то в результате у нейрона с весами w_i мы получим выход $y_i = f(w_i^T x)$, где f – его функция активации.

Мы можем представить вычисление, которое делают все нейроны сразу, вот так:

Векторизация: упрощаем сложное

&&

Если в слое k нейронов, и веса у них $w_1, w_2, ..., w_k$, где $w_i = (w_{i1}, w_{i2}, ..., w_{in})^T$, а на вход подаётся вектор $x = (x_1, x_2, ..., x_n)^T$, то в результате у нейрона с весами w_i мы получим выход $y_i = f(w_i^T x)$, где f – его функция активации.

Мы можем представить вычисление, которое делают все нейроны сразу, вот так:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_k \end{pmatrix} = y = f(Wx) = \begin{pmatrix} f(w_1^T x) \\ \vdots \\ f(w_k^T x) \end{pmatrix}, \text{ где } W = \begin{pmatrix} w_1 \\ \vdots \\ w_k \end{pmatrix} = \begin{pmatrix} w_{11} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{k1} & \dots & w_{kn} \end{pmatrix}$$

Векторизация: упрощаем сложное

&&

Если в слое k нейронов, и веса у них $w_1, w_2, ..., w_k$, где $w_i = (w_{i1}, w_{i2}, ..., w_{in})^T$, а на вход подаётся вектор $x = (x_1, x_2, ..., x_n)^T$, то в результате у нейрона с весами w_i мы получим выход $y_i = f(w_i^T x)$, где f – его функция активации.

Мы можем представить вычисление, которое делают все нейроны сразу, вот так:

$$\begin{pmatrix} y_1 \\ \vdots \\ y_k \end{pmatrix} = y = f(Wx) = \begin{pmatrix} f(w_1^T x) \\ \vdots \\ f(w_k^T x) \end{pmatrix}, \text{ где } W = \begin{pmatrix} w_1 \\ \vdots \\ w_k \end{pmatrix} = \begin{pmatrix} w_{11} & \dots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{k1} & \dots & w_{kn} \end{pmatrix}$$

и вычисление всего слоя сведётся к умножению матрицы весов на вектор входов, а затем покомпонетному применению одной и той же функции активации. Это оказывается сильно эффективнее ещё за счёт того, что видеокарты способны оптимизировать и кратно ускорить процесс такого умножения.

Функции активации

Sigmoid &&

Исторически было принято “активировать” нейрон с помощью сигмоиды:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

она ограничена, стремится к 0 при $x \to -\infty$ и к единице при $x \to \infty$.

Исторически было принято “активировать” нейрон с помощью сигмоиды:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

она ограничена, стремится к 0 при $x \to -\infty$ и к единице при $x \to \infty$. Её производную легко посчитать как $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, доказывать это мы конечно же не будем.

Sigmoid &&

Исторически было принято “активировать” нейрон с помощью сигмоиды:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

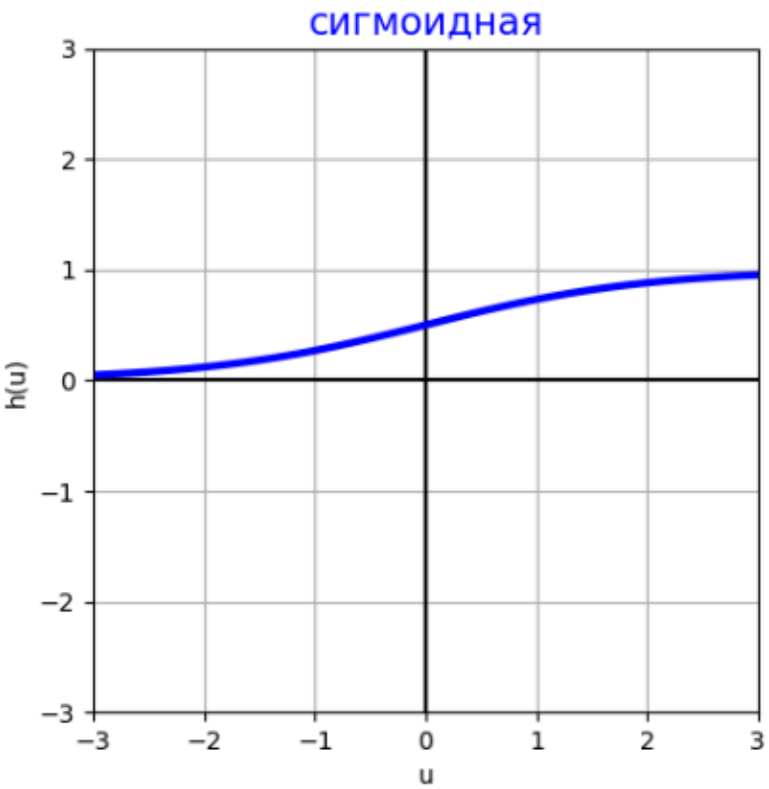
она ограничена, стремится к 0 при $x \to -\infty$ и к единице при $x \to \infty$. Её производную легко посчитать как $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, доказывать это мы конечно же не будем.

При больших (или очень маленьких) значениях x производная сигмоиды практически равна нулю. В глубоких сетях, когда мы перемножаем эти маленькие производные , градиент становится нулём до того, как дойдет до первых слоев $\Rightarrow$ НС никак не будет обучаться.

Sigmoid

&&

$$h(u) = \sigma(u) = \frac{1}{1 + e^{-u}}$$



Tanh

&&

Сигмоида имеет насыщение в точке 0, т.е. если пытаться обучить значение этой функции в ноль, вход будет стремиться к $-\infty$, а производная к нулю, что затрудняет градиентный спуск. Частично решить эту проблему призван гиперболический тангенс:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh

&&

Сигмоида имеет насыщение в точке 0, т.е. если пытаться обучить значение этой функции в ноль, вход будет стремиться к $-\infty$, а производная к нулю, что затрудняет градиентный спуск. Частично решить эту проблему призван гиперболический тангенс:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Для сравнения: $\tanh'(0) = 1$, а $\sigma'(0) = \frac{1}{4}$.

Tanh

&&

Сигмоида имеет насыщение в точке 0, т.е. если пытаться обучить значение этой функции в ноль, вход будет стремиться к $-\infty$, а производная к нулю, что затрудняет градиентный спуск. Частично решить эту проблему призван гиперболический тангенс:

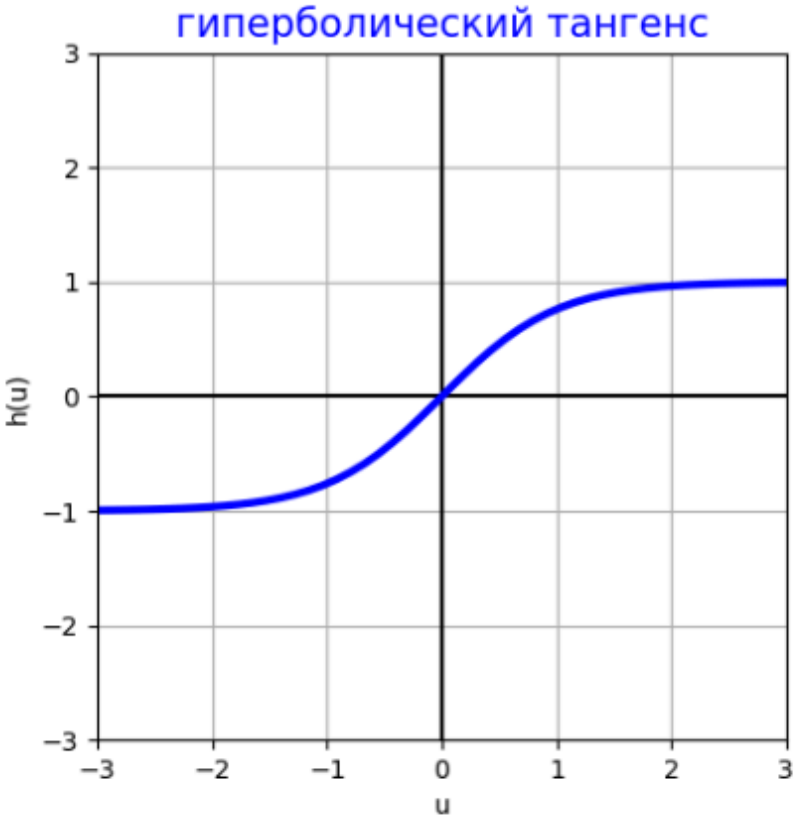
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Для сравнения: $\tanh'(0) = 1$, а $\sigma'(0) = \frac{1}{4}$. Справедливости ради, проблема затухания на хвостах по прежнему остаётся, но теперь она не такая критическая.

Tanh

&&

$$h(u) = \text{tangh}(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} = 2\sigma(2u) - 1$$



Однако tanh по-прежнему быстро выходит на асимптоты $+-1$, что может испортить градиентный спуск.

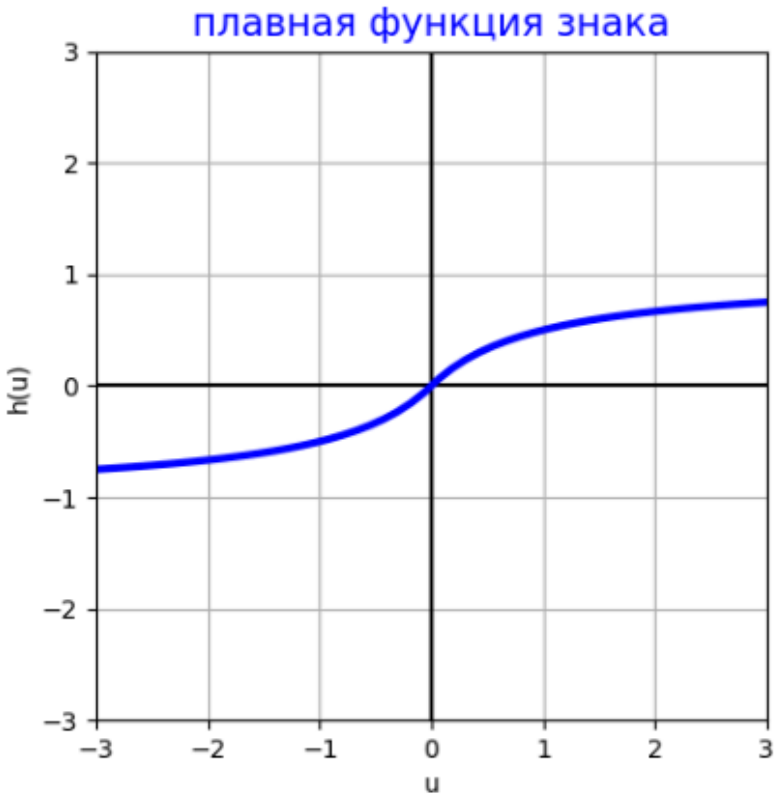
$$SF(x) = \frac{x}{1 + |x|}$$

SoftSign идейно повторяет tanh, но имеет характер приближения к асимптотам +1 и −1 полиномиальный, а не экспоненциальный, вследствие чего на значения $+-1$ она выходит медленнее, что ускоряет сходимость при настройке сети.

SoftSign

&&

$$h(u) = \frac{u}{1 + |u|}$$



Rectified Linear Units (ReLU) &&

$$ReLU(x) = \max(0, x)$$

Имеет нетривиальный градиент +1 на полуоси $x > 0$, за счёт чего используется как в скрытых слоях, так и в выходных слоях, где нужно предсказывать неотрицательный регрессионный отклик.

Rectified Linear Units (ReLU) &&

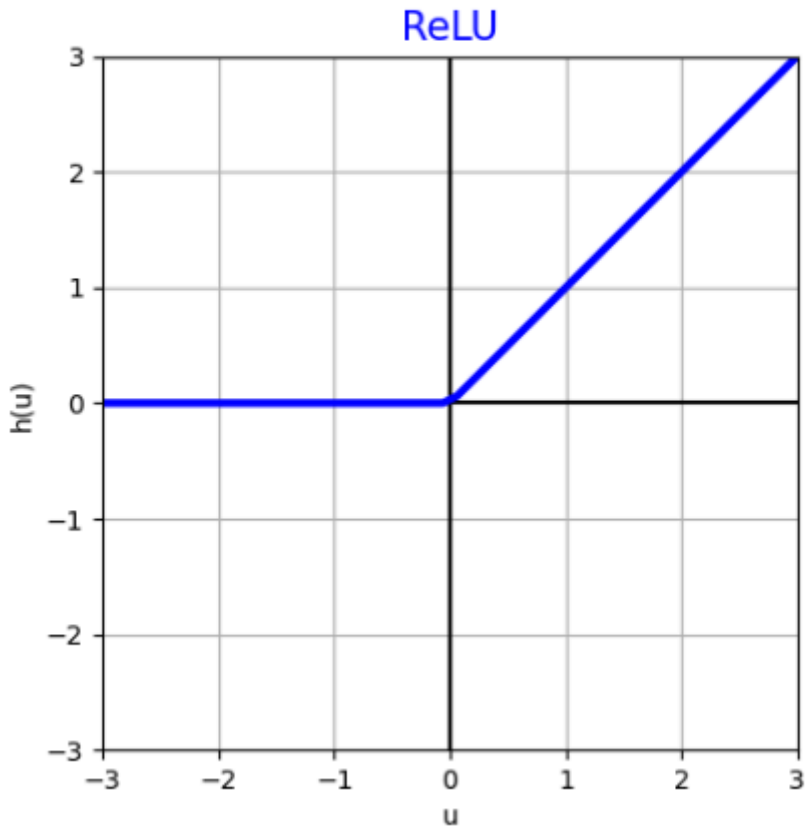
$$ReLU(x) = \max(0, x)$$

Имеет нетривиальный градиент +1 на полуоси $x > 0$, за счёт чего используется как в скрытых слоях, так и в выходных слоях, где нужно предсказывать неотрицательный регрессионный отклик.

Dead ReLU: Если на нейрон прилетает большой отрицательный градиент, веса могут обновиться так, что он всегда будет выдавать 0 для любых входов. Такой нейрон “умирает” и больше никогда не обновится, так как его градиент в этой области – чистый ноль.

Rectified Linear Units (ReLU) &&

$$h(u) = \max\{0; u\}$$



Leaky ReLU &&

Именно для решения этой проблемы придумали Leaky ReLU, который исключает холостые вычисления:

$$h(x) = \max(\alpha x, x), \alpha \in (0, 1)$$

Leaky ReLU

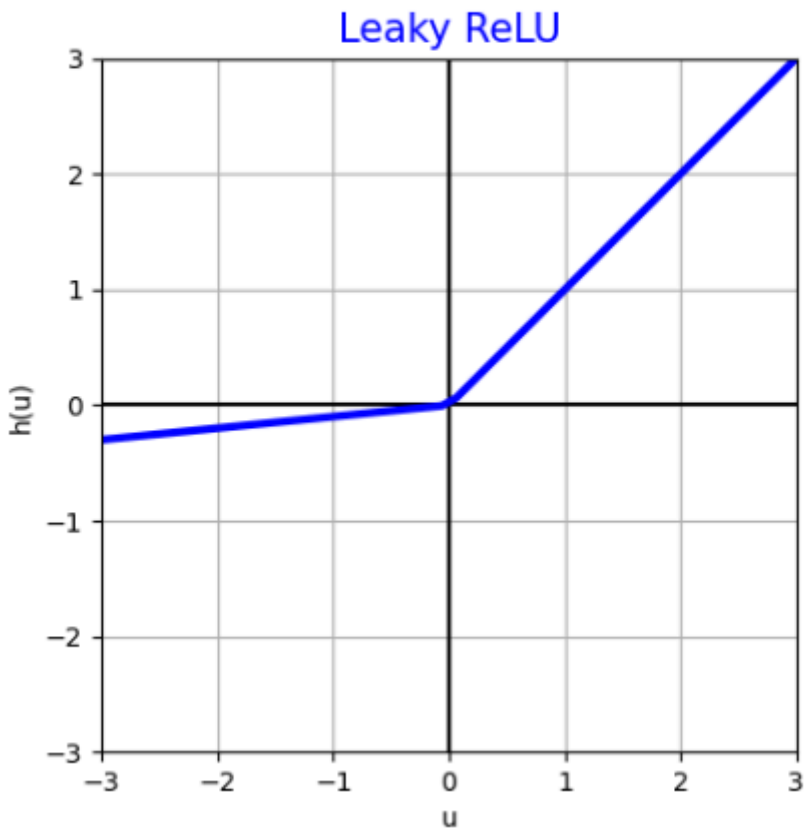
&&

Именно для решения этой проблемы придумали Leaky ReLU, который исключает холостые вычисления:

$$h(x) = \max(\alpha x, x), \alpha \in (0, 1)$$

Активация Leaky ReLU обладает всеми достоинствами ReLU-активации, но кроме того выдаёт ненулевые значения и для отрицательных аргументов, за счёт чего не вырождается в ноль и не приводит к лишним вычислениям вхолостую.

Leaky ReLU

&&

$h(u) = \max\{\alpha u; u\}$, $\alpha \in (0, 1)$ – гиперпараметр